

Role

Act as a senior Databricks Unity Catalog architect and Python software architect.

Design and implement a production-grade Databricks notebook/job utility that migrates existing Unity Catalog table-level Row Filters (RLS) and Column Masks to Unity Catalog ABAC policies.

The utility must be modular, idempotent, auditable, extensible, and safe to run across large numbers of catalogs, schemas, and tables.

Business Requirement

Currently, some Unity Catalog tables have:

1. Row-level security implemented directly as table Row Filters using existing functions.
2. Column-level security implemented directly as table Column Masks using existing functions.

The target state is ABAC.

For each table:

- Identify the existing table-level Row Filter/RLS function.
- Identify existing table-level Column Mask functions.
- Create/configure the corresponding ABAC policy using the existing security function.
- Apply the ABAC policy to the table.
- Validate that the ABAC policy is correctly applied.
- Only after successful validation should the existing table-level Row Filter / Column Mask be removed.
- Preserve the original configuration in audit/rollback metadata.

Do not change the underlying security function unless explicitly required by the Databricks ABAC API/model.

The migration must preserve the existing security semantics.

Scope

The utility must support these execution scopes:

1. All catalogs
2. Selected catalogs
3. All schemas within selected catalogs
4. Selected schemas within selected catalogs
5. Specific tables for testing/troubleshooting

Support schema exclusion using regex.

Example:

```
catalogs = ["catalog1", "catalog2"]

schemas = {
    "catalog1": ["schema1", "schema2"],
    "catalog2": ["schema3"]
}

exclude_schema_regex = "test_.*|tmp_.*"
```

The scope resolver must be completely separate from the migration engine.

Execution Modes

Implement at least:

```
INVENTORY
MIGRATE
INVENTORY_AND_MIGRATE
VERIFY
RECONCILE
ROLLBACK
```

Also support:

```
dry_run = true/false
continue_on_error = true/false
max_parallelism = N
```

INVENTORY MODE

Inventory must make no changes.

For every table in scope, identify:

- catalog
- schema
- table
- table type
- whether Row Filter exists

- Row Filter expression
- Row Filter function
- whether Column Masks exist
- masked columns
- mask expressions
- mask functions
- whether ABAC policies already exist
- migration eligibility
- reason for non-eligibility
- current migration status

Example conceptual output:

```
catalog.schema.table
  row_filter:
    exists = true
    function = security.customer_rls

  column_masks:
    email = security.email_mask
    phone = security.phone_mask

  abac:
    exists = false

  eligibility:
    ELIGIBLE
```

Inventory must be persisted to an audit/inventory table.

CORE ARCHITECTURE REQUIREMENT

The most important requirement is that the lowest-level table conversion capability must be a pluggable module.

Define a clean interface conceptually similar to:

```
convert_table(table_name, options) -> ConversionResult
```

The caller must not need to know how the conversion is implemented.

The module must:

1. Inspect the table.
2. Identify existing RLS and/or Column Masks.
3. Validate referenced functions.
4. Determine required ABAC policy.

5. Create/apply ABAC policy.
6. Validate resulting state.
7. Remove old table-level security only after successful validation.
8. Return a structured result.

The result must contain at least:

```
status
table_name
rls_status
column_mask_status
source_functions
target_policies
error_code
error_message
started_at
completed_at
validation_status
rollback_metadata
```

Possible status values:

```
SUCCESS
FAILED
SKIPPED
ALREADY_MIGRATED
NOT_ELIGIBLE
```

PLUGIN ARCHITECTURE

Do not implement RLS and Column Mask migration as one large function.

Create pluggable migration components:

```
MigrationPlugin
├── RLSMigrationPlugin
└── ColumnMaskMigrationPlugin
```

Each plugin should conceptually expose:

```
discover(table)
validate(table)
convert(table)
```

```
verify(table)
rollback(table)
```

The table conversion engine invokes the applicable plugins.

This must allow additional security migration types to be added in the future without changing the core orchestration engine.

RLS MIGRATION

For an existing table Row Filter:

```
catalog.schema.table
|
+-- Row Filter
    |
    +-- existing security function
```

The migration should create/apply the corresponding ABAC policy using the existing function.

Do not recreate or modify the function unless required.

Validate:

- function exists
- function is accessible
- function is compatible with the target ABAC mechanism
- policy was successfully created
- policy was successfully applied
- resulting table metadata reflects the expected ABAC configuration

Only then remove the old table-level Row Filter.

COLUMN MASK MIGRATION

A table can have multiple masked columns.

Example:

```
customer
  email      -> security.mask_email
  phone      -> security.mask_phone
  credit_card -> security.mask_card
```

The design must support multiple column masks per table.

Each masked column/function mapping must be inventoried and migrated independently.

Do not assume one mask function per table.

Only remove the old mask after the corresponding ABAC configuration has been successfully validated.

IDEMPOTENCY

The utility must be safe to execute multiple times.

Example:

```
First execution:
table A -> SUCCESS

Second execution:
table A -> ALREADY_MIGRATED
```

Never blindly create duplicate policies.

Before migration:

1. Inspect current UC state.
2. Determine whether equivalent ABAC configuration already exists.
3. Compare expected vs actual state.
4. Migrate only if required.

The Unity Catalog state must remain the source of truth.

The audit table must not be treated as the authoritative state.

DRIFT DETECTION

Support reconciliation.

Example:

```
Audit table:
  migration_status = SUCCESS
```

Actual UC:
ABAC policy no longer exists

The utility must identify this as configuration drift.

Expose:

DRIFT

as a reconciliation result.

SAFETY

Never do:

DROP OLD RLS
↓
CREATE ABAC

Use:

READ EXISTING SECURITY CONFIGURATION
↓
VALIDATE SOURCE
↓
CREATE ABAC
↓
APPLY ABAC
↓
VALIDATE ABAC
↓
REMOVE OLD RLS/MASK
↓
VALIDATE FINAL STATE

The migration should minimize any possibility of temporarily weakening security.

DRY RUN

Implement dry-run mode.

Example:

```
dry_run = true
```

must not modify Unity Catalog.

Instead produce:

```
Table:
  sales.prod.customer

Current:
  RLS = security.customer_rls

Proposed:
  ABAC policy = <deterministic policy identity>
  Function = security.customer_rls

Actions:
  CREATE ABAC
  APPLY ABAC
  REMOVE TABLE RLS

Result:
  WOULD_MIGRATE
```

AUDIT / STATE MANAGEMENT

Create a persistent audit/state table.

Include fields such as:

```
run_id
catalog
schema
table
object_type
source_security_type
source_function
source_definition
target_policy
target_definition
status
error_code
error_message
validation_status
rollback_metadata
```



```
started_at
completed_at
```

Also maintain inventory information separately if useful.

The audit design must support:

- restartability
- reporting
- troubleshooting
- reconciliation
- rollback
- migration history

ROLLBACK

Design rollback support.

Before modifying anything, capture sufficient metadata to restore:

- original Row Filter
- original Column Masks
- original function references
- ABAC policies created by this utility

Rollback must only remove ABAC configuration created by this migration utility.

Never delete or modify an unrelated pre-existing ABAC policy.

POLICY STRATEGY

Do not hard-code policy naming/creation strategy deep inside the converter.

Create a policy strategy abstraction.

For example:

```
PolicyStrategy
├─ FunctionBasedPolicyStrategy
└─ TableBasedPolicyStrategy
```

The selected strategy should determine the deterministic identity/name of the target ABAC policy.

The design must handle cases where the same security function is used by many tables.

ORCHESTRATION

The orchestrator should work like:

```
Resolve Scope
  |
  v
Discover Tables
  |
  v
Inventory
  |
  v
Filter Eligible Tables
  |
  v
For each table
  |
  +--> TableConversionEngine
        |
        +--> RLS Plugin
        |
        +--> Column Mask Plugin
  |
  v
Validate
  |
  v
Persist Result
  |
  v
Generate Summary
```

The orchestrator should support parallel table processing, but the lowest-level table conversion function must remain independently callable.

One table failure must not necessarily stop the complete migration when:

```
continue_on_error = true
```

CODE STRUCTURE

Implement the project with clean separation similar to:

```

abac_migration/
├── config/
│   └── config_loader.py
├── scope/
│   └── scope_resolver.py
├── discovery/
│   ├── catalog_discovery.py
│   ├── schema_discovery.py
│   ├── table_discovery.py
│   ├── rls_discovery.py
│   └── mask_discovery.py
├── inventory/
│   ├── inventory_manager.py
│   └── inventory_repository.py
├── migration/
│   ├── migration_engine.py
│   ├── table_converter.py
│   └── plugins/
│       ├── base_plugin.py
│       ├── rls_to_abac.py
│       └── mask_to_abac.py
├── validation/
│   ├── pre_validation.py
│   ├── post_validation.py
│   └── drift_detection.py
├── rollback/
│   └── rollback_manager.py
├── audit/
│   └── audit_repository.py
└── notebook/
    └── abac_migration_driver

```

However, because this must run as a Databricks notebook/job, do NOT create a standalone VM/CLI application.

The notebook must be the execution entry point.

Keep the notebook thin and put reusable logic into Python modules where Databricks notebook/job packaging permits it.

DATBRICKS IMPLEMENTATION

Use Databricks/Unity Catalog supported APIs and metadata interfaces.

Do not invent REST API endpoints, SQL syntax, or ABAC syntax.

Where the exact Databricks API behavior is uncertain, explicitly mark it as an implementation dependency/TODO rather than fabricating an API.

The implementation must be compatible with the target Databricks Runtime/API version supplied through configuration.

Prefer supported Unity Catalog APIs/metadata mechanisms over parsing UI output.

CONFIGURATION

Support configuration through notebook/job parameters.

Example:

```
mode
scope_type
catalogs
schemas
tables
exclude_schema_regex
dry_run
continue_on_error
max_parallelism
audit_catalog
audit_schema
audit_table
policy_strategy
```

Secrets/credentials should not be hard-coded.

REPORTING

At the end of execution provide:

```
Total tables discovered
Total tables with RLS
```

```
Total tables with Column Masks
Total tables with both
Total eligible
Total migrated
Total already migrated
Total skipped
Total failed
Total drifted
Total not eligible
```

Also provide per-table details.

Example:

```
SUCCESS
FAILED
SKIPPED
ALREADY_MIGRATED
NOT_ELIGIBLE
DRIFT
```

TESTABILITY

The lowest-level converter must be unit-testable without requiring the entire orchestration framework.

Mock the Unity Catalog/API layer.

Provide tests for at least:

1. Table with RLS only.
 2. Table with Column Mask only.
 3. Table with both.
 4. Table with no security functions.
 5. Missing function.
 6. Existing ABAC policy.
 7. Already migrated table.
 8. ABAC creation failure.
 9. ABAC validation failure.
 10. Old RLS removal failure.
 11. Multiple masked columns.
 12. Dry run.
 13. Retry/idempotent execution.
 14. Configuration drift.
 15. Rollback.
-

IMPORTANT DESIGN PRINCIPLES

1. Modular architecture.
 2. Pluggable table conversion module.
 3. RLS and Column Mask migration must be separate plugins.
 4. Inventory must be independent from migration.
 5. Scope resolution must be independent from migration.
 6. Idempotent execution.
 7. Dry-run support.
 8. Audit everything.
 9. Unity Catalog remains the source of truth.
 10. Never remove existing security before validating the replacement.
 11. Support rollback.
 12. Continue-on-error processing.
 13. Support large catalog/schema/table scopes.
 14. Do not hard-code customer-specific catalog/schema names.
 15. Do not invent Databricks APIs.
 16. The notebook is the execution entry point.
 17. Keep the notebook thin.
 18. The lowest-level function must accept a table name and return a structured success/failure result.
 19. The lowest-level converter must be independently pluggable/reusable.
 20. Design the system so additional security migration plugins can be added later.
-

Expected Deliverables

Before writing implementation code, first provide:

1. High-level architecture.
2. Component responsibilities.
3. End-to-end execution flow.
4. Data model for inventory/audit.
5. Plugin interfaces.
6. Conversion state machine.
7. Idempotency strategy.
8. Rollback strategy.
9. Error-handling strategy.
10. Notebook/job parameter design.
11. Proposed project/file structure.
12. API/Databricks dependency list.
13. Risks and edge cases.
14. Test strategy.

Then implement the utility incrementally.

Do not build one large notebook containing all logic.

The core requirement is:

```
table_name
|
v
Pluggable Table Converter
|
+--> detect RLS
+--> detect masks
+--> validate functions
+--> create/apply ABAC
+--> validate
+--> remove old security
+--> validate final state
|
v
ConversionResult
```

That component must remain independent of catalog-wide orchestration.